

Create, Deploy, and Test an HTTP Trigger Azure Function Using Visual Studio Code

Summary:-

In this lab, an **HTTP Trigger Azure Function** is created using **Visual Studio Code** and deployed to **Azure Functions**. You will create a Function App using the **Flex Consumption** hosting plan, configure a local Azure Functions project with the **.NET 10 isolated worker runtime**, generate an HTTP Trigger function with **Anonymous authorization**, deploy the function to Azure, and test it using the Azure Portal to verify successful execution.

Let's Start:-

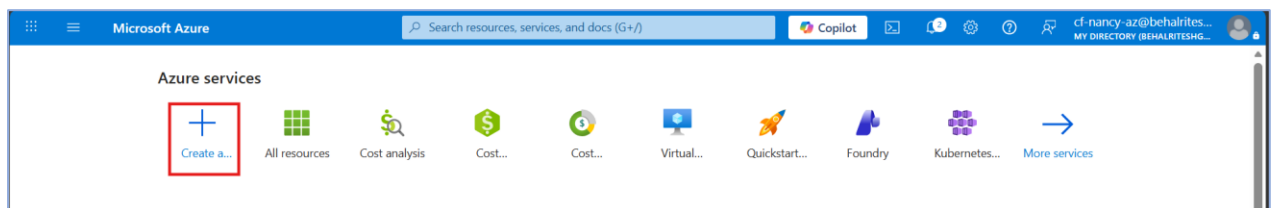
After completing this lab, you will be able to:

- Create an Azure Function App using the Flex Consumption hosting plan.
- Configure an Azure Functions project in Visual Studio Code using the .NET 10 isolated worker runtime.
- Create an HTTP Trigger Azure Function.
- Configure Anonymous authorization for the function.
- Deploy the Azure Function project to Azure.
- Test the deployed function using the Azure Portal.
- Verify successful execution of the Azure Function.

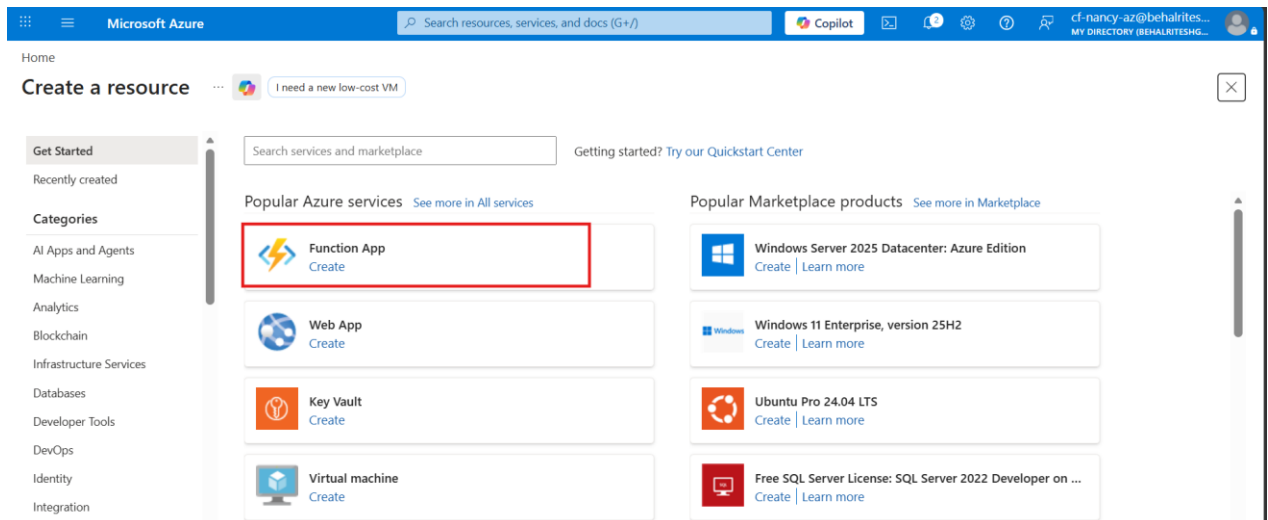
Lab Steps:-

Step 1 – Create a Function App

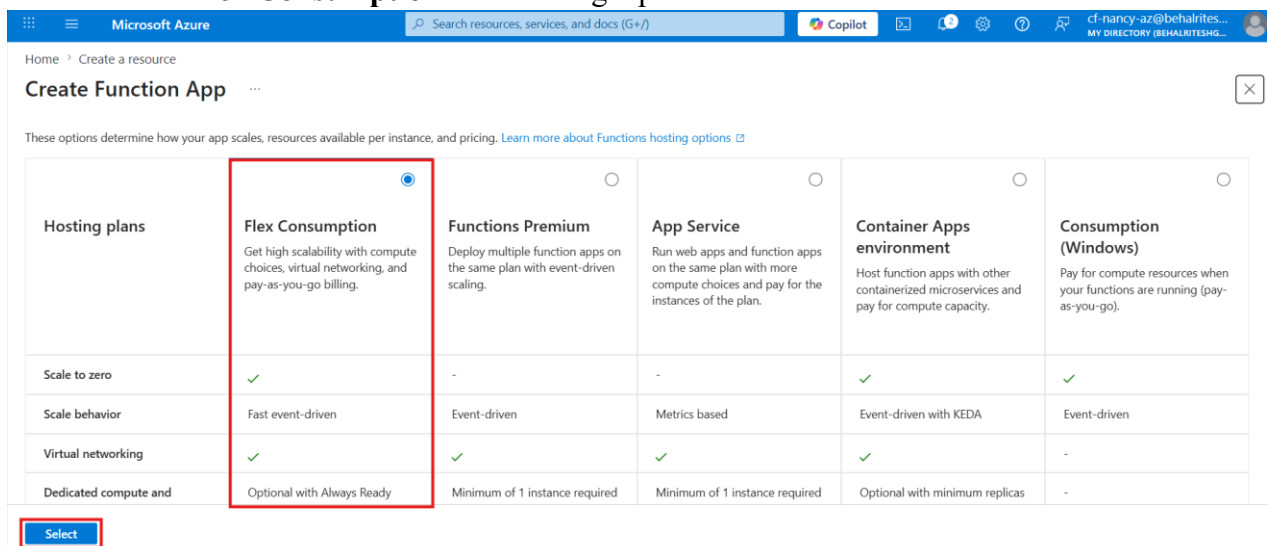
- Sign in to **Azure Portal**.
- Click **Create a Resource**.



- Select **Function App**. Click **Create**.



- Select **Flex Consumption** as a Hosting Option. Click Select.



- Configure the **Function App** using the following settings.
 - Subscription : **Select** your Azure Subscription.
 - Leave Resource Group as **Default**.
 - Function App Name : Enter a unique name

Microsoft Azure

Search resources, services, and docs (G+J)

Copilot

cf-nancy-az@behalrites...
MY DIRECTORY (BEHALRITESHG...)

Home > Create a resource > Create Function App

Create Function App (Flex Consumption)

Basics Storage Azure OpenAI Networking Monitoring Durable Functions Deployment Authentication Tags Review + create

Create a function app, which lets you group functions as a logical unit for easier management, deployment and sharing of resources. Functions lets you execute your code in a serverless environment without having to first create a VM or publish a web application.

Project Details

Select a subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *

Resource Group *
[Create new](#)

Instance Details

Function App name *
-gah9dngjfravdea4.canadacentral-01.azurewebsites.net

[Review + create](#) < Previous Next: Storage >

- Select:
 - Region : East US
 - Runtime stack : .NET
 - Version : 10 (LTS),isolated worker model
 - Instance memory : Default (2048)

Microsoft Azure

Search resources, services, and docs (G+J)

Copilot

cf-nancy-az@behalrites...
MY DIRECTORY (BEHALRITESHG...)

Home > Create a resource > Create Function App

Create Function App (Flex Consumption)

Basics Storage Azure OpenAI Networking Monitoring Durable Functions Deployment Authentication Tags Review + create

Basics

Region *

Runtime stack *

Version *
⚠ NET 8 (LTS), isolated worker model will reach EOL on 10/11/2026 [Learn More](#)

Instance size *

Zone redundancy

Instances of your app are distributed across availability zones for increased reliability. [More about zone redundancy.](#)

Zone redundancy

☐ Enabled: Your Flex Consumption app will be zone redundant. This changes your app's required instance count per function or function group.

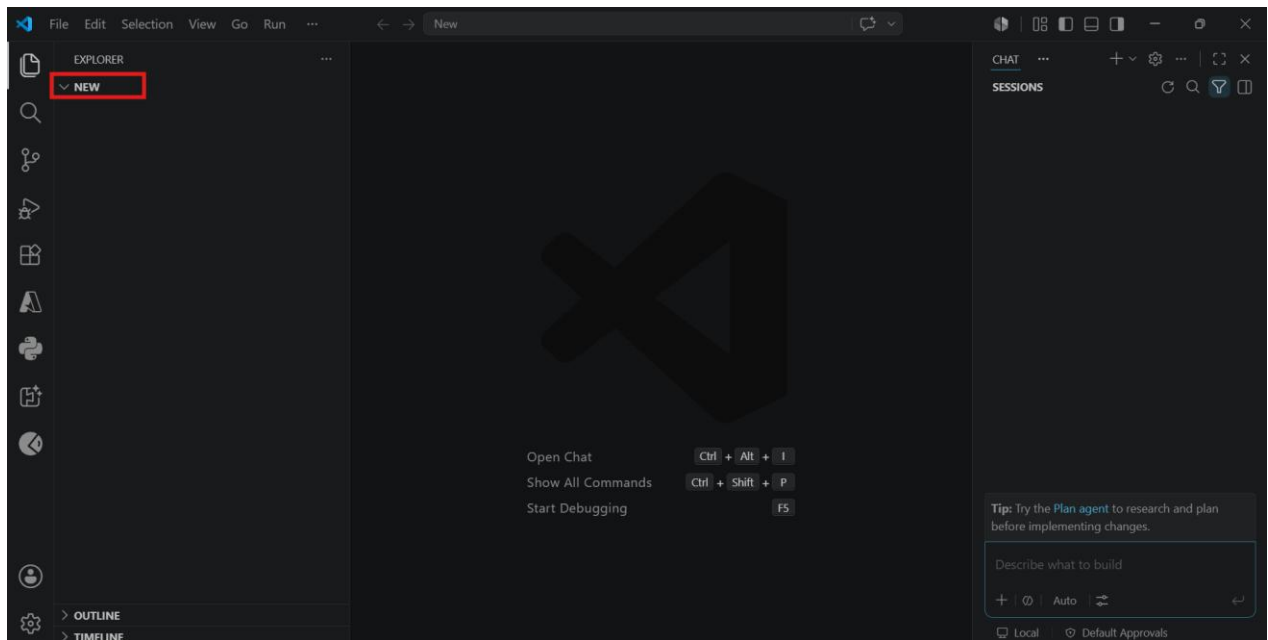
☒ Disabled: Your Flex Consumption app will not be zone redundant.

[Review + create](#) < Previous Next: Storage >

- Leave all remaining settings as their default values.
- Click **review + create**. Then **Create**.

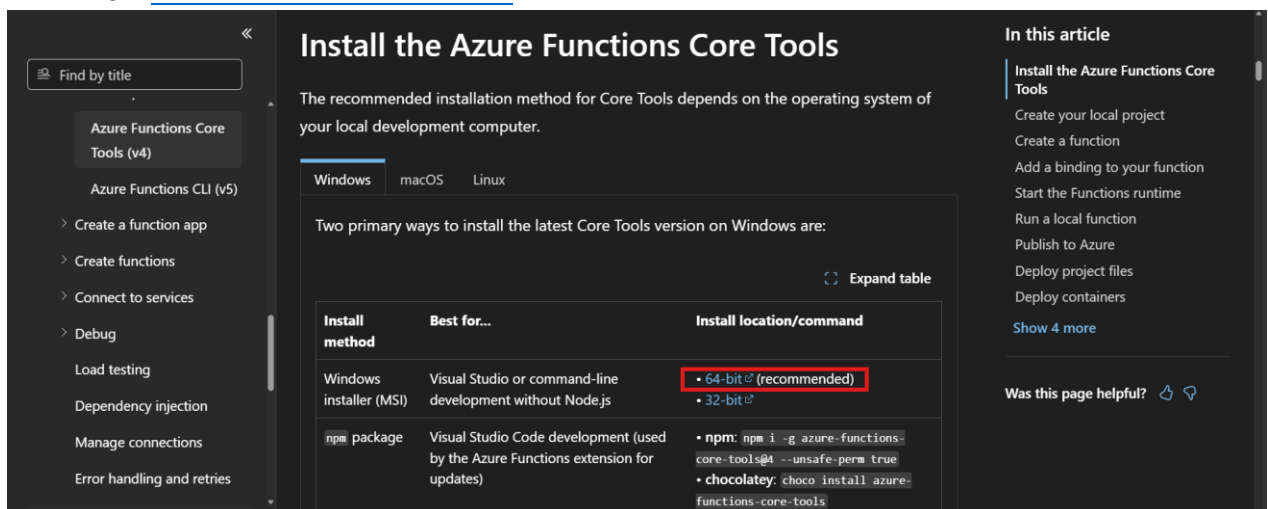
Step 2 – Open an Empty Project Folder

- Open **Visual Studio Code**.
- Select **File** → **Open Folder**.
- Create or select an **empty folder** for the Azure Functions project.

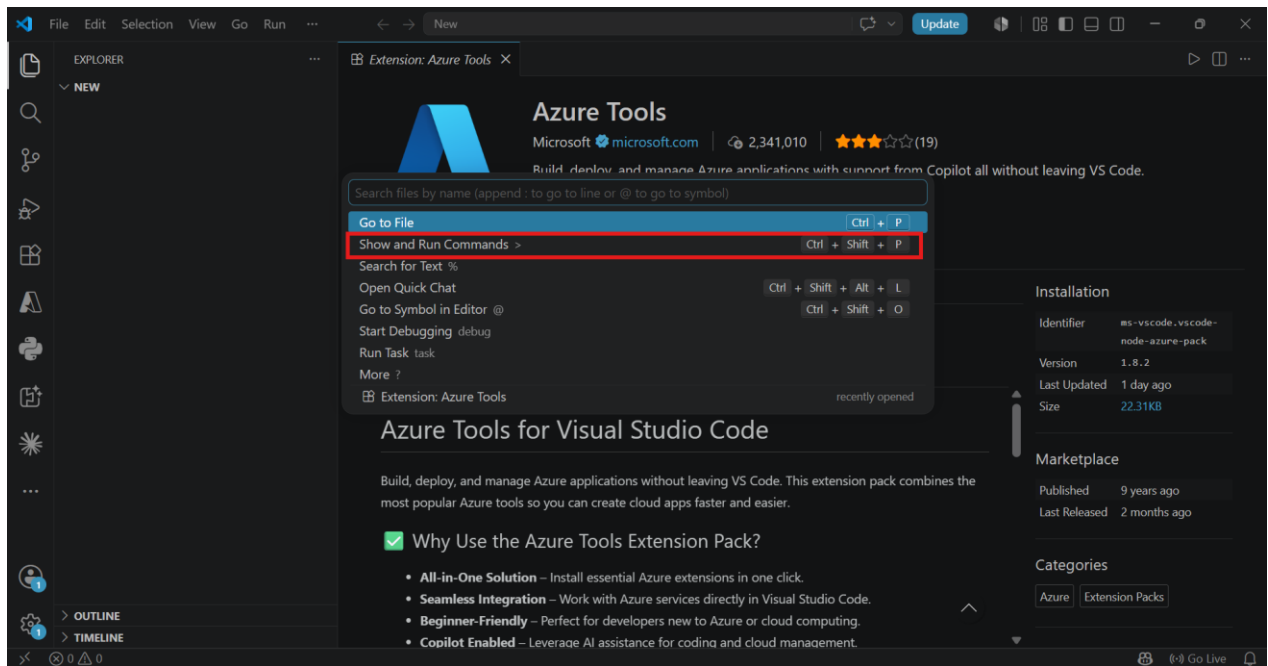


Step 3 – Open the Command Palette

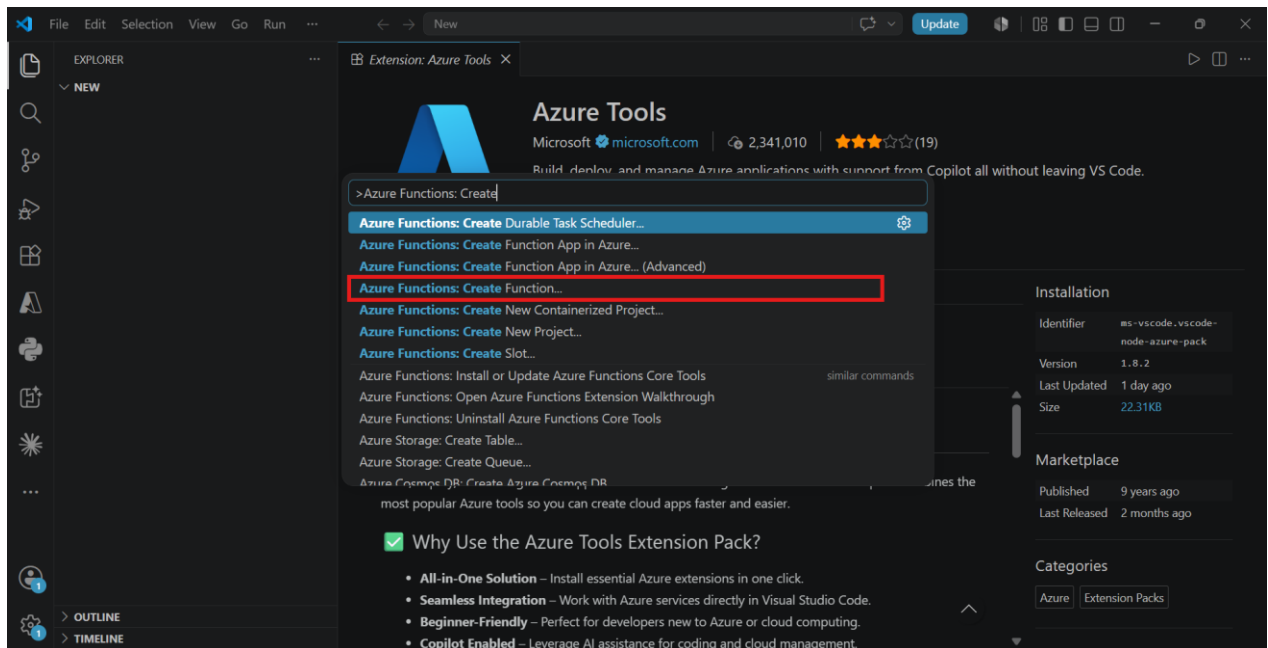
- Make sure you have:
 - Extension: Azure Tools, Azure Functions
 - .NET 10 SDK installed
 - [Azure Functions Core Tools](#) installed



- Press **Ctrl + Shift + P**.
- Click on **Show and Run Commands**.



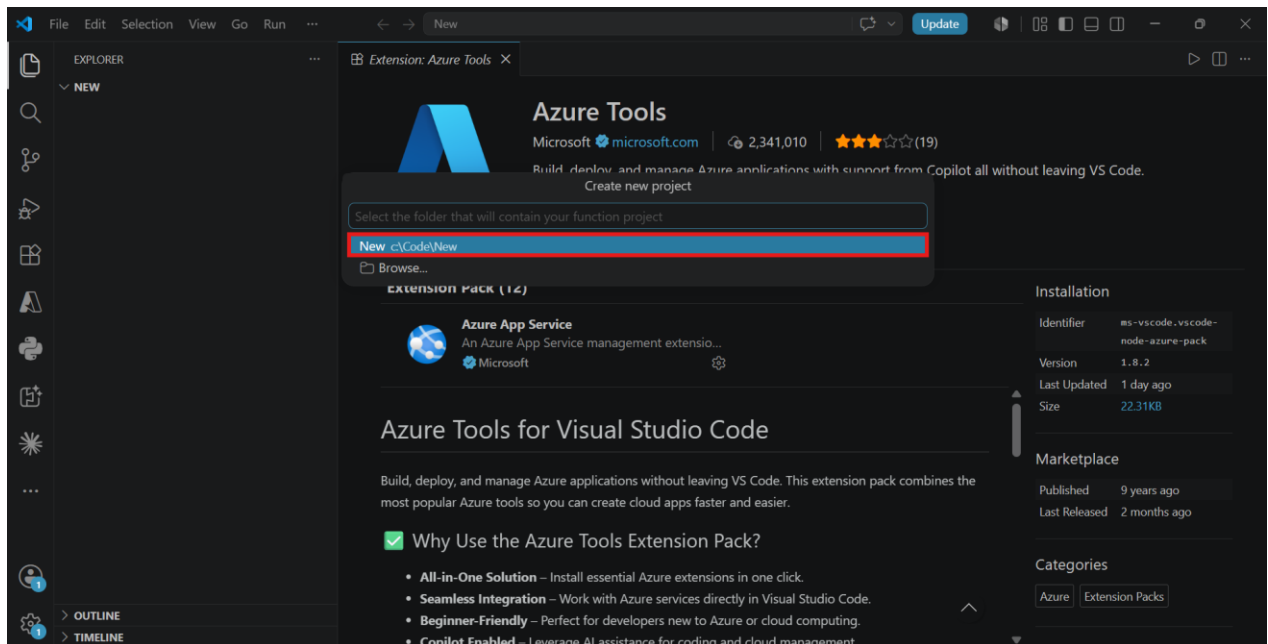
- **Search for Azure Functions: Create Function.**



- Select the command.

Step 4 – Select the Project Folder

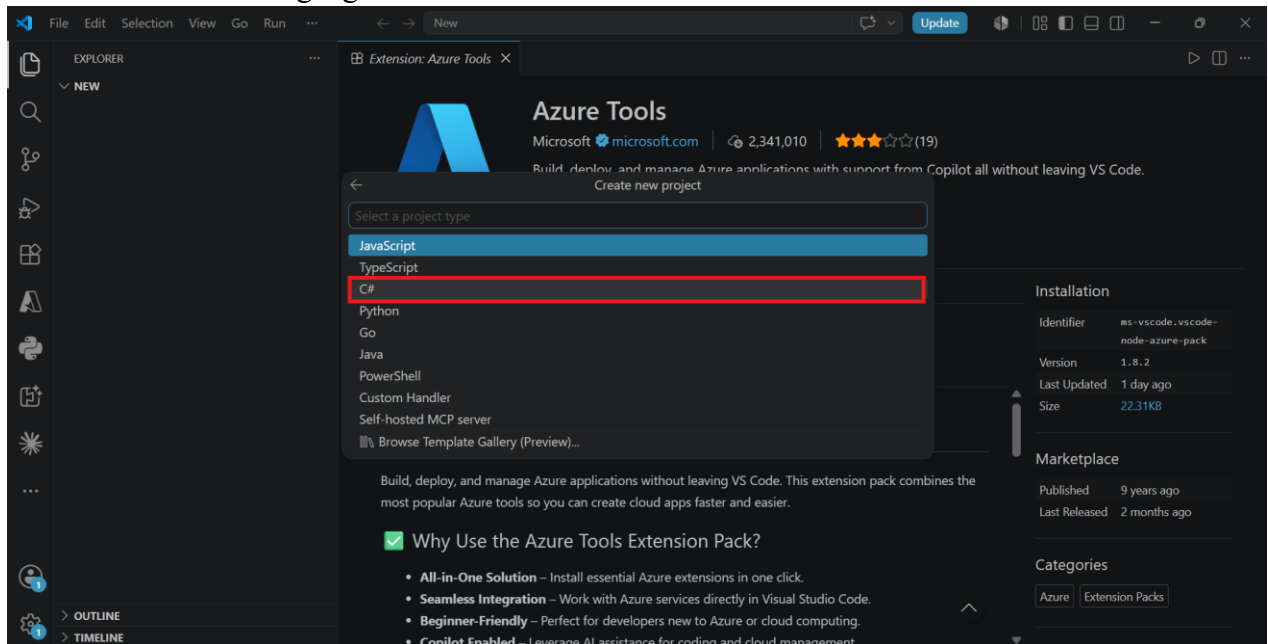
- Select the empty folder opened in the previous step.



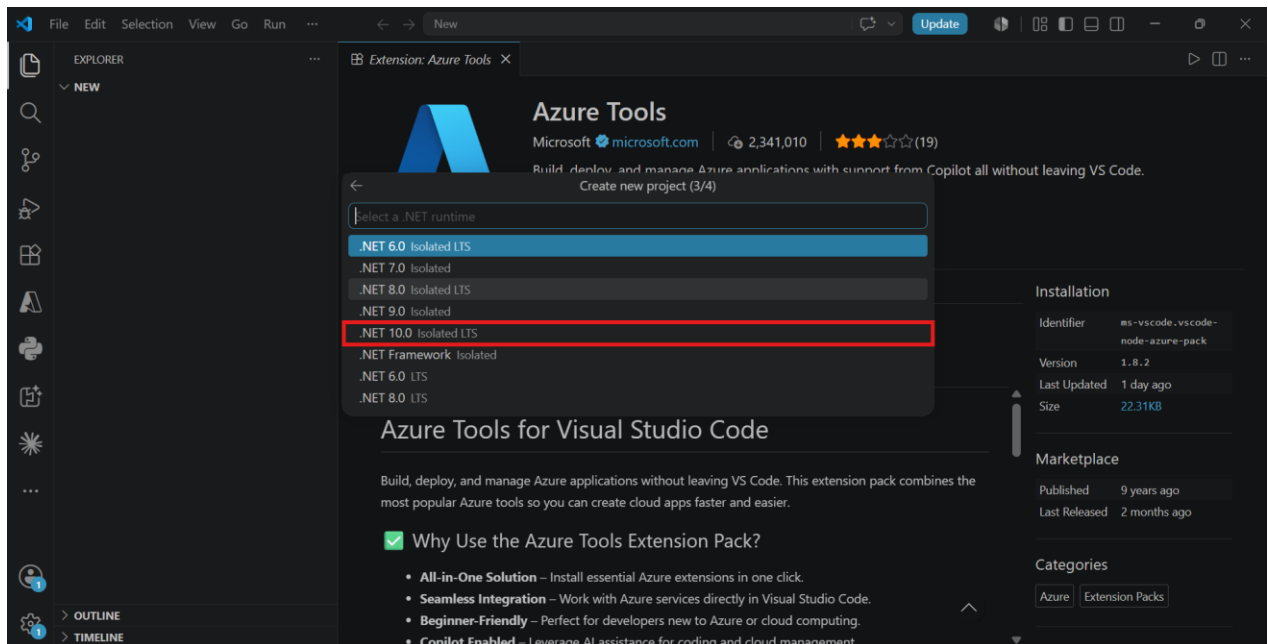
- Click Select Folder.

Step 5 – Configure the Azure Functions Project

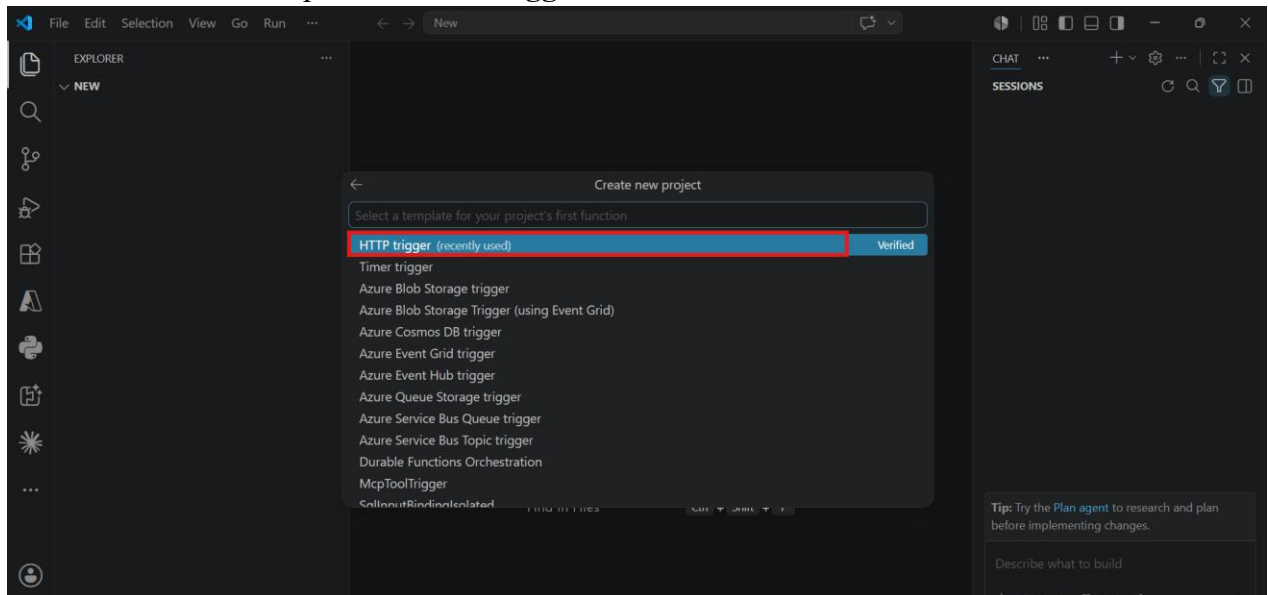
- Select the following options when prompted.
 - Language : C#



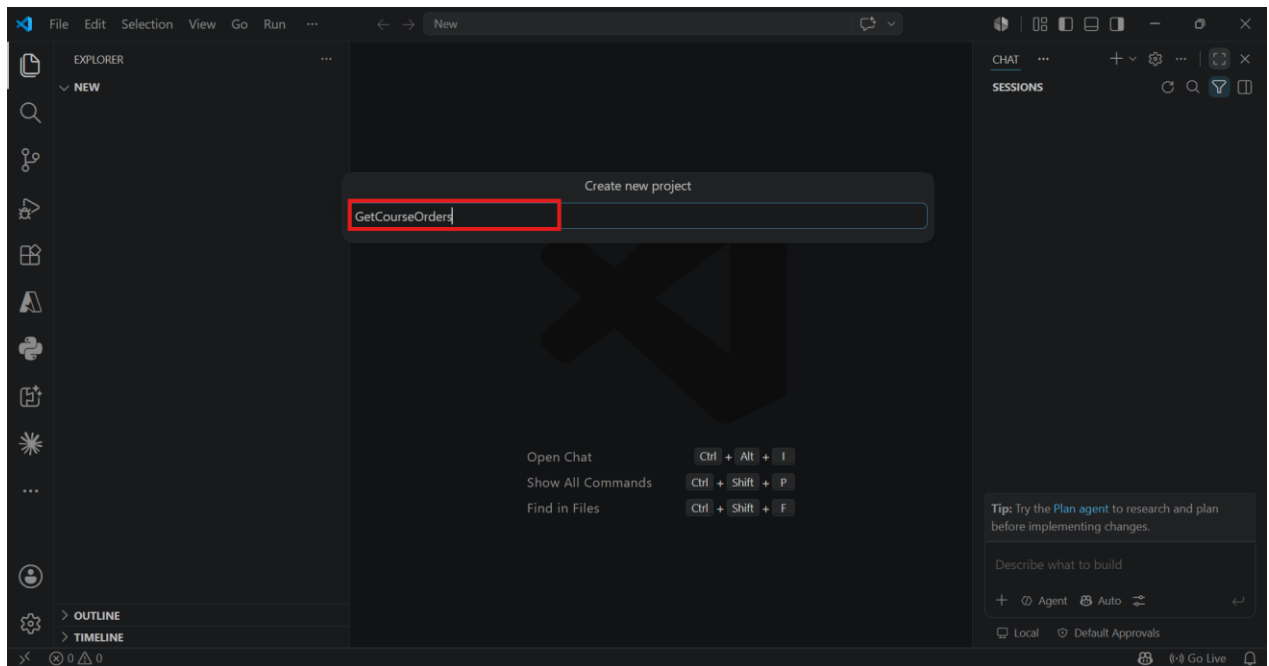
- Runtime : .NET 10



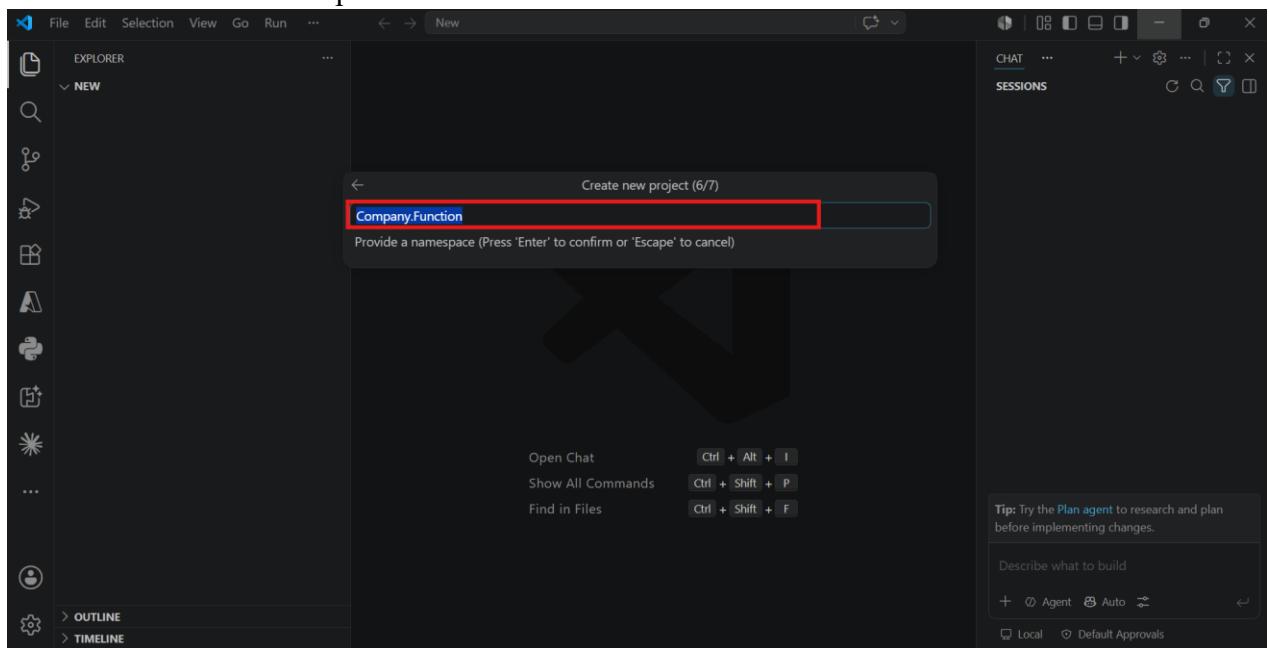
○ Template : **HTTP Trigger**.



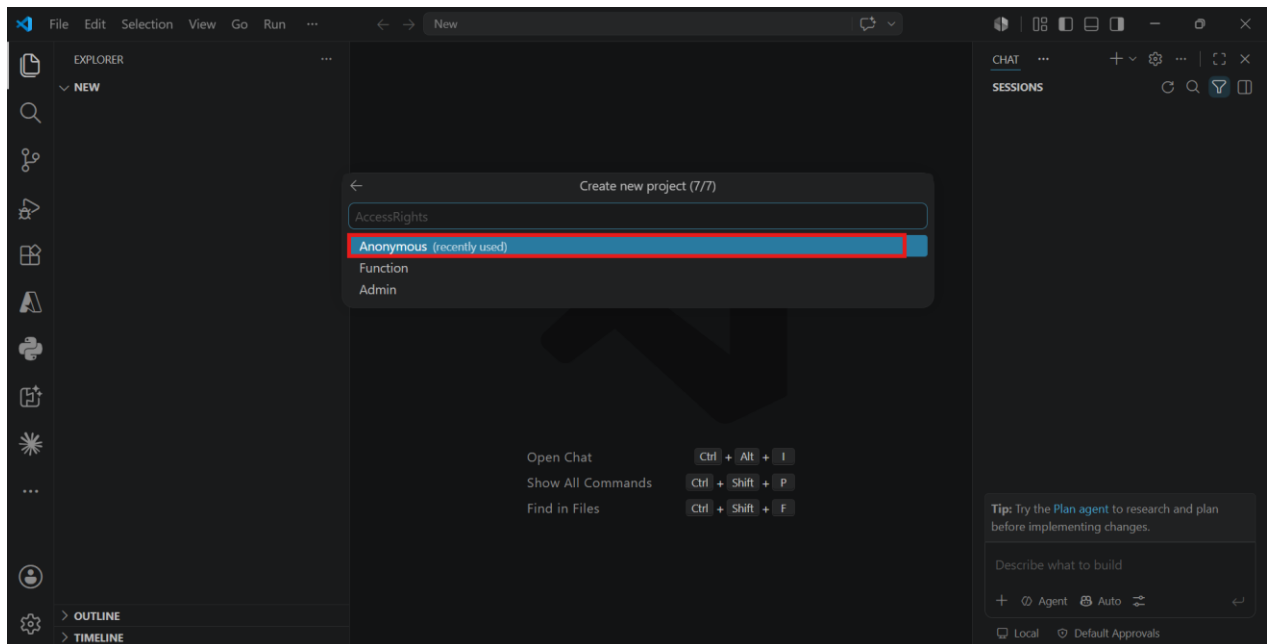
○ Rename it as **GetCourseOrders**. Click **Enter**.



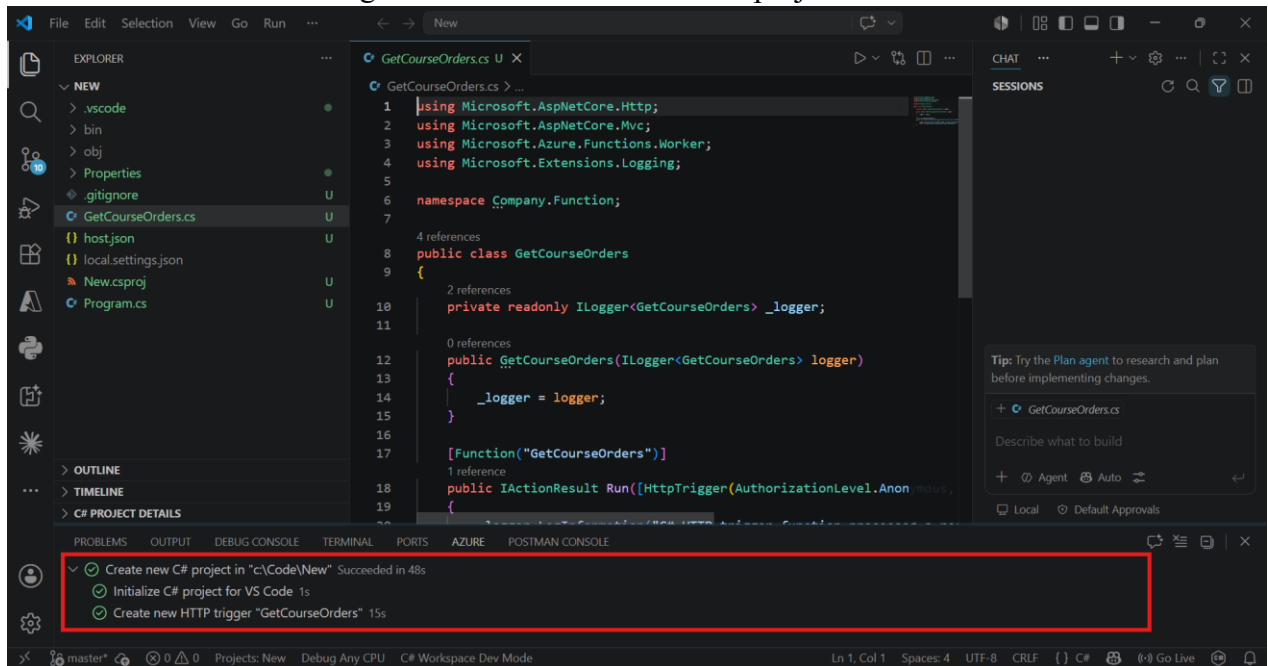
○ Namespace : Leave the default value. Click **Enter**.



○ Authorization Level : Anonymous



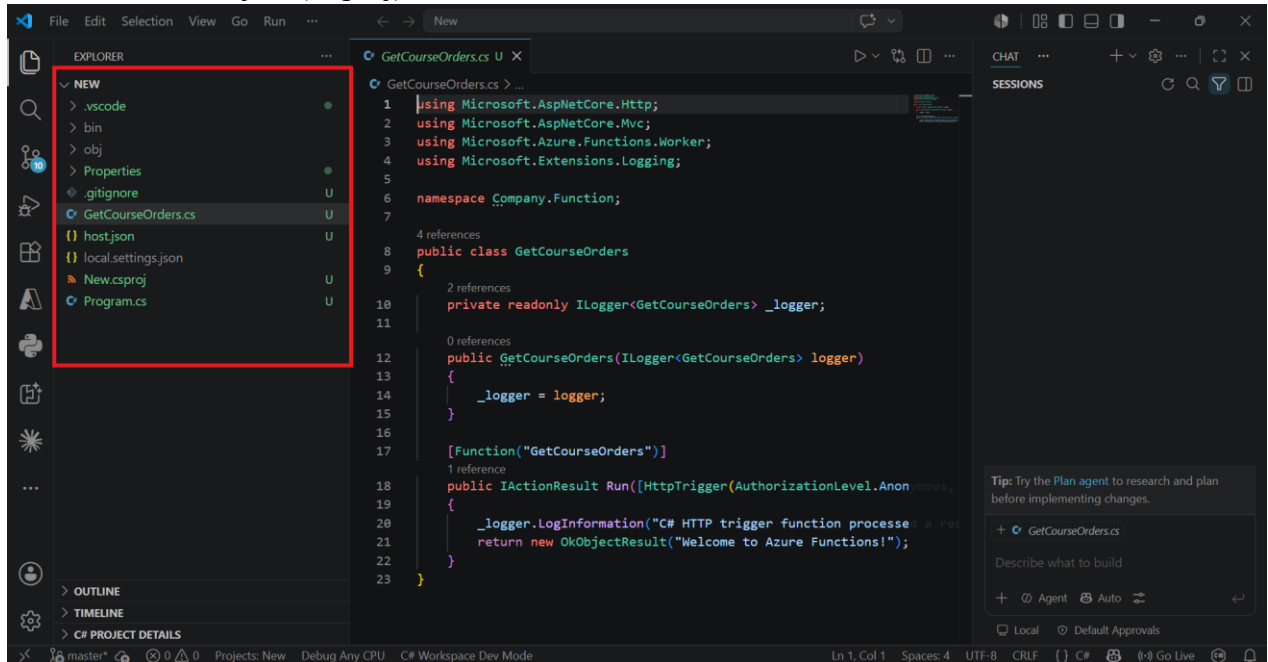
- Visual Studio Code generates the Azure Functions project.



Step 6 – Review the Generated Project

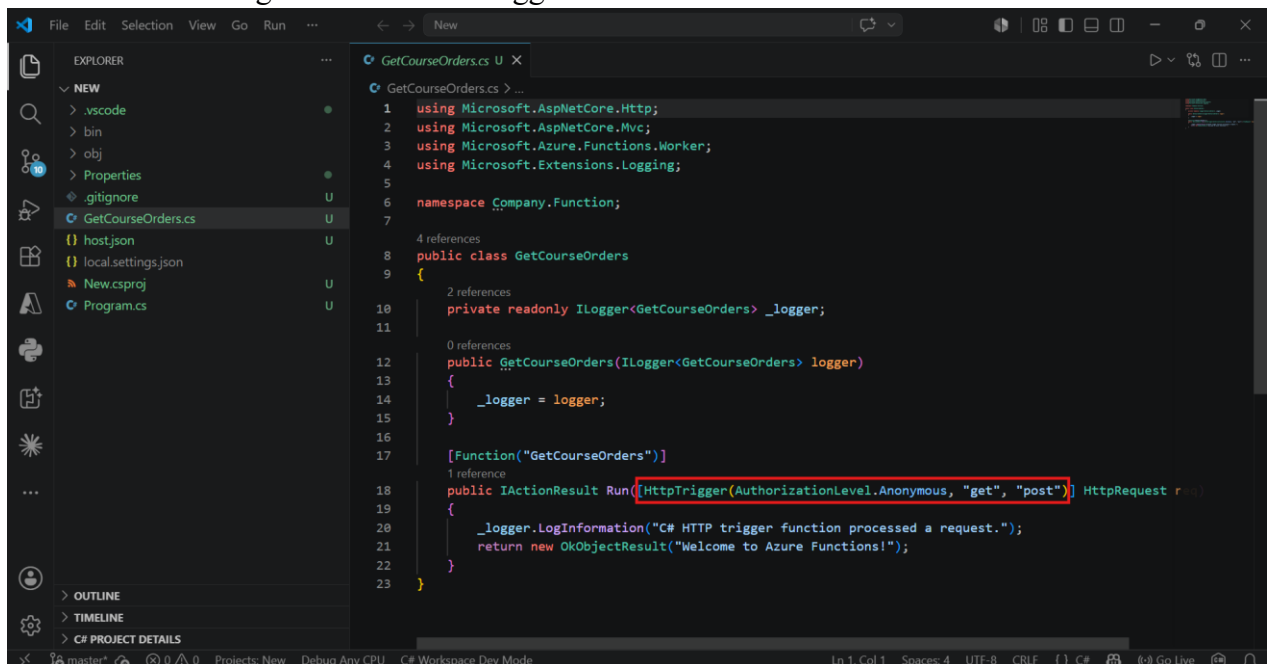
- Expand the project in the **Explorer** pane.
- **Verify** that the project contains files similar to the following:
 - **GetCourseOrders.cs**
 - **host.json**
 - **local.settings.json**
 - **Program.cs**

- Project (.csproj) file



Step 7 – Review the Function Code

- Open GetCourseOrders.cs.
- Review the generated HTTP Trigger function.

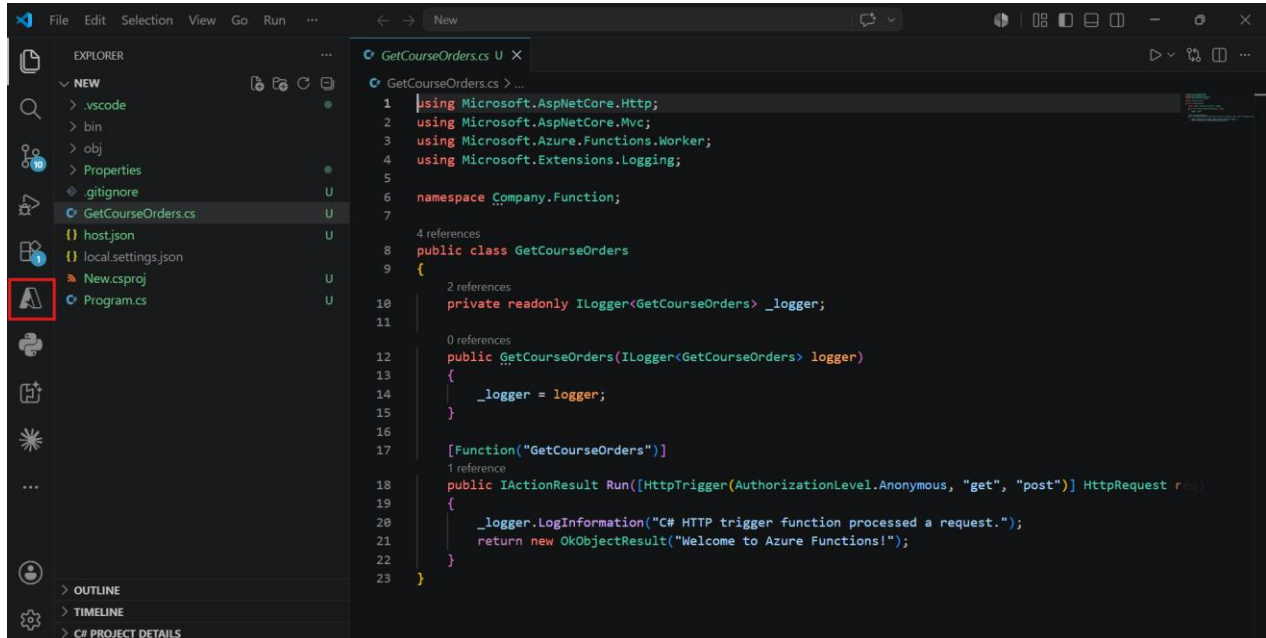


- Verify that the trigger is configured with:
 - HTTP Trigger
 - GET and POST methods (default template)
 - Anonymous authorization

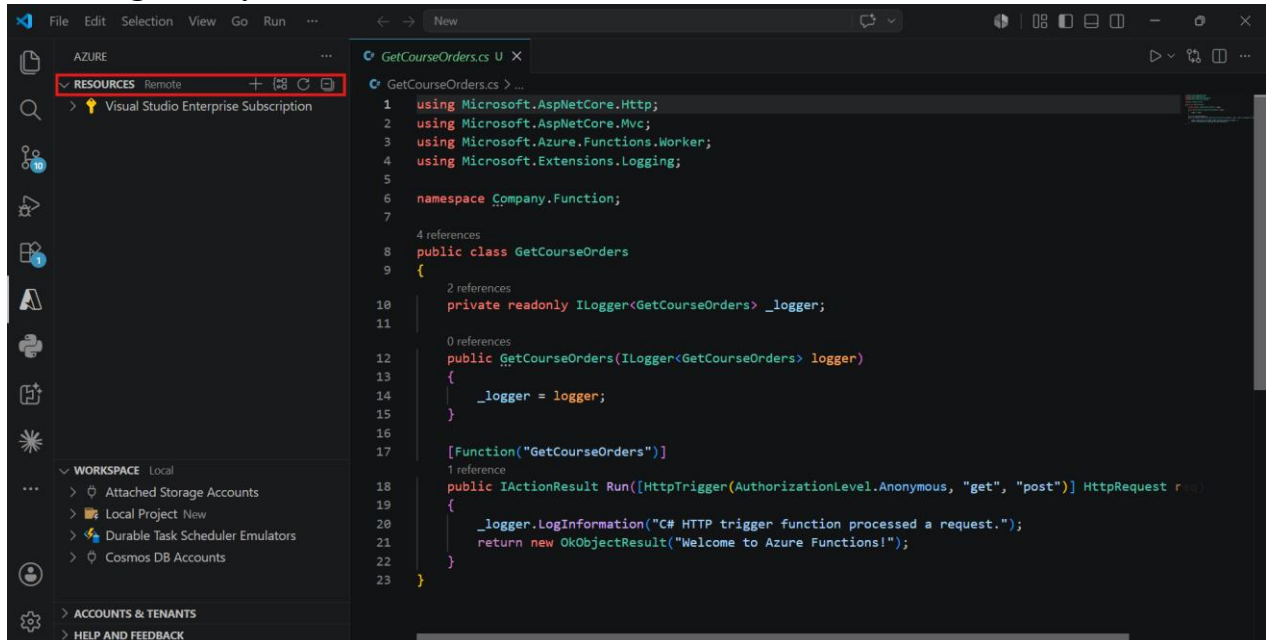
- Save all project files. The Azure Function project is now ready for local execution and deployment.

Step 8 – Deploying the Function Code

- Go to the left panel. Select **Azure** Icon or Press Ctrl + Shift + A.

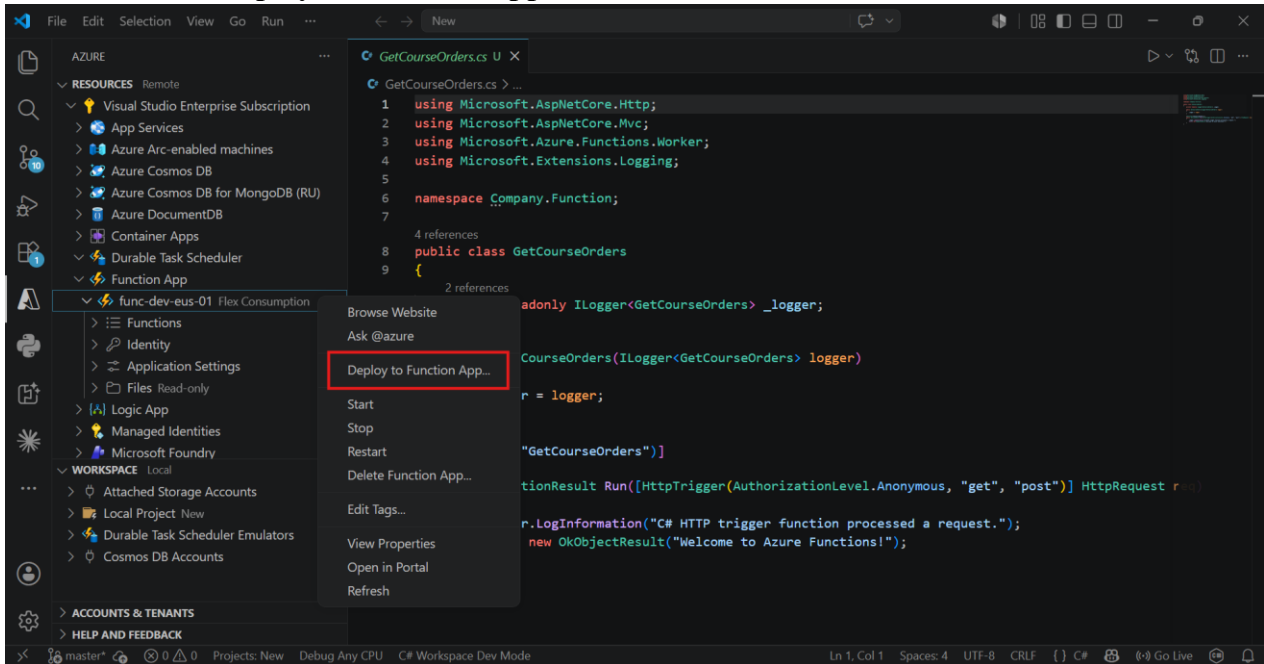


- Expand Resources.
- Sign in to your Azure account.

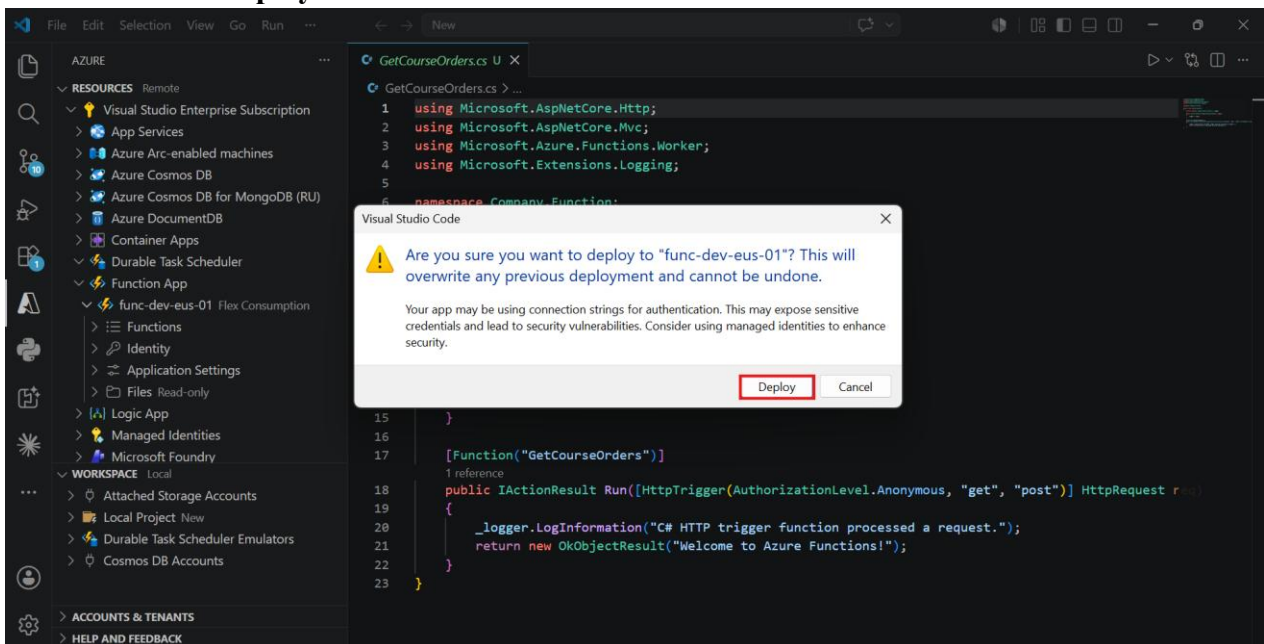


- Expand your Subscription Bar.
- Select **Function App** and expand it.
- Select your **Function App** name and Right – click.

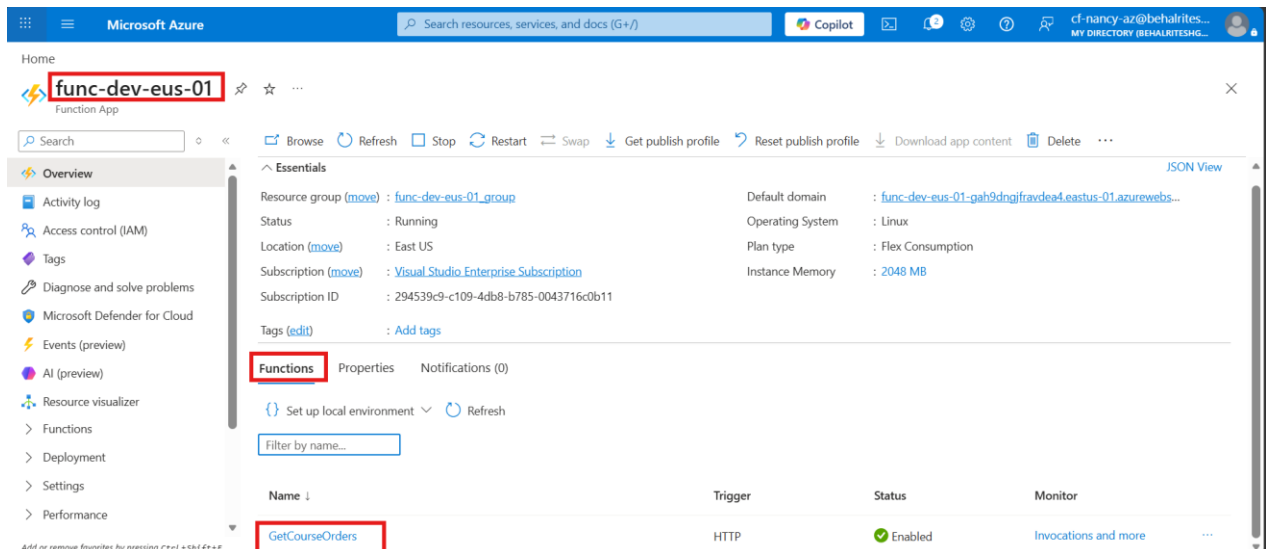
- Click on **Deploy to Function App**.



- Click on **Deploy**.



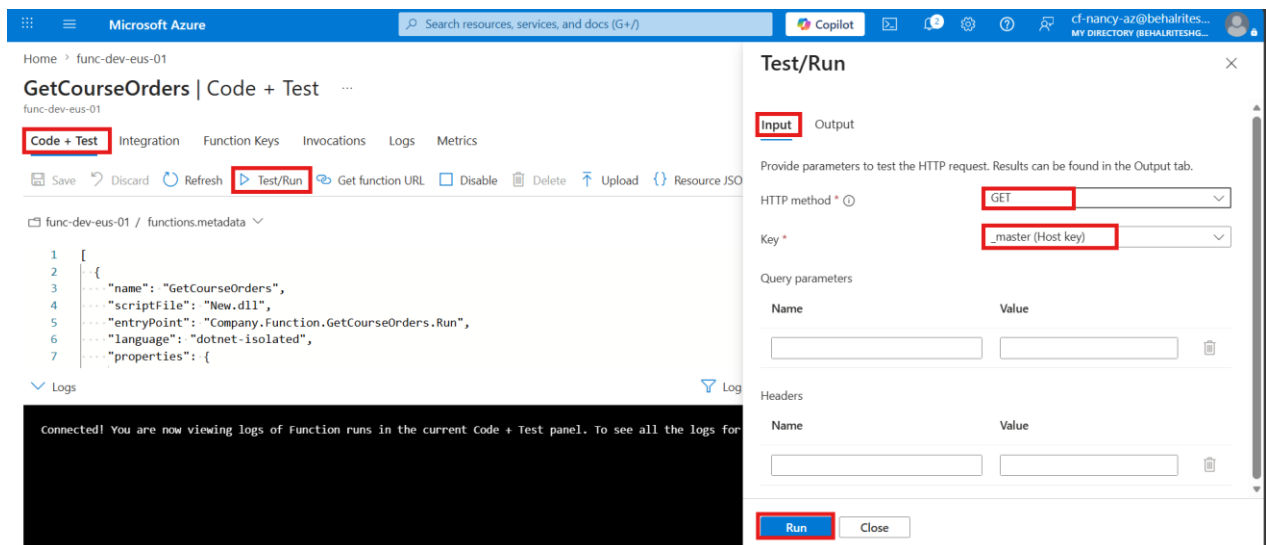
- Verify that the deployment published the Function Code onto the created Function App (e.g. func-dev-eus-01) under the **Functions** Tab.



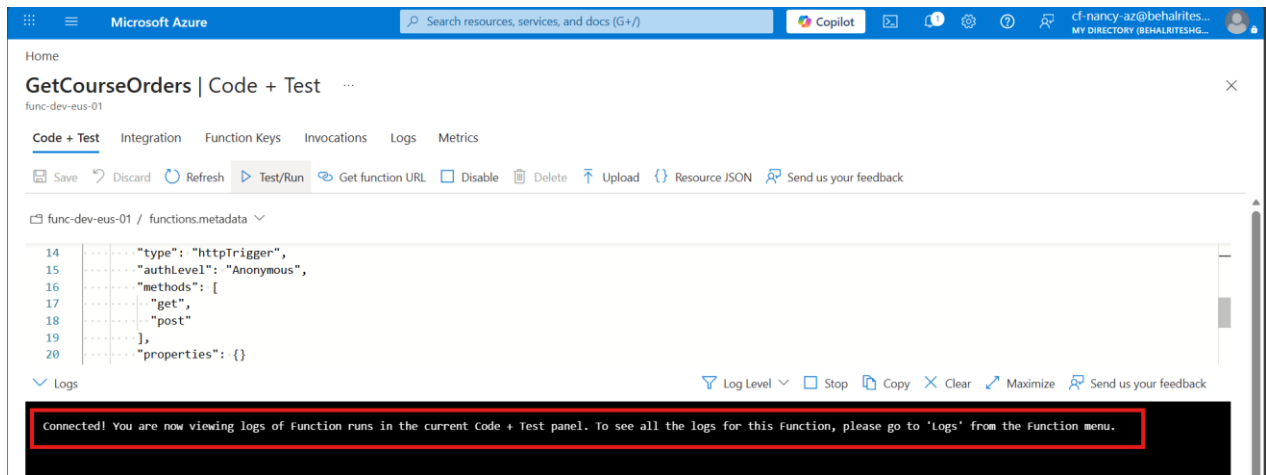
- Verify the status is enabled.

Step 8 – Testing the Function Code

- Click on **GetCourseOrders**.
- Go to the **Code + Test** tab. Click **Test/Run**.
- Now, go to Input field and Select:
 - **GET** as HTTP method
 - **_master (Host key)** as Key.
- Click **Run**.



- Verify:
 - HTTP response code returns **200 OK**.
 - HTTP response content returns **"Welcome to Azure Functions!"**.
 - Logs return **"Connected!..."**.



Verification

Verify the following:

- The Azure Function App is deployed successfully and is accessible from the Azure Portal.
- The **GetCourseOrders** function is listed under the **Functions** section and its status is **Enabled**.
- The function executes successfully using the **Test/Run** option in the Azure Portal.
- The function returns an **HTTP 200 (OK)** response.
- The response body displays the expected default message: **"Welcome to Azure Functions!"**.
- The execution completes without errors and the function logs confirm successful execution.